EKF Design: IMU Dead-Reckoning + Wi-Fi FTM Fusion

Group 2 – MobileAP

April 8, 2026

Contents

1 Running the App **2**
1.1 Requirements . . . 2
1.2 Step-by-Step Usage . . . 2
1.3 FTM Ranging Calibration . . . 2

2 Overview **3**

3 State Vector **3**

4 Coordinate Frames **3**

5 Body-to-World Frame Rotation **4**

6 Predict Step – Kinematic IMU Model with Body-Frame Bias **4**
6.1 Bias Correction Order . . . 4
6.2 State Transition . . . 4
6.3 State Transition Jacobian . . . 5
6.4 Process Noise Covariance . . . 5
6.5 Covariance Propagation . . . 6

7 Update Step 1 – Wi-Fi FTM Range **6**
7.1 Measurement Model . . . 6
7.2 Jacobian . . . 6
7.3 Kalman Update . . . 7
7.4 FTM Pre-processing . . . 7

8 Update Step 2 – ZUPT (Zero-Velocity Pseudo-Measurement) **7**
8.1 Threshold Calibration . . . 7
8.2 Detection . . . 7
8.3 Measurement Model . . . 7
8.4 Kalman Update . . . 8

9 Initialisation **8**

10 Sequence of Operations (per IMU event) **8**

1 Running the App

1.1 Requirements

- Android device with Wi-Fi RTT (IEEE 802.11mc) support (Android 9+, API 28+). Check with `WifiRttManager.isAvailable()`.
- A 5 GHz access point that responds to FTM ranging requests. If the AP does not advertise `is80211mcResponder`, add its BSSID to the `KNOWN_FTM_BSSIDS` set in `FusionActivity.kt` and the app will construct a `ResponderConfig` manually (API 33+ only).
- Permissions granted at runtime: `ACCESS_FINE_LOCATION` and, on API 33+, `NEARBY_WIFI_DEVICES`.

1.2 Step-by-Step Usage

1. **Set AP position.** Enter the AP's known coordinates $(x_{\mathrm{AP}}, y_{\mathrm{AP}})$ and the height difference Δz between the phone and the AP (default 1.2 m) in the "AP:" row, then tap **Set**.
2. **Set phone start position (optional).** Enter the phone's starting coordinates in the "Phone:" row. If left blank the EKF initialises with a large position uncertainty and reports only range until a start position is supplied later.
3. **Scan and select AP.** Tap **Scan**. FTM-capable APs discovered in the scan results (or matching the hard-coded BSSID list) appear in the spinner. Select the target AP.
4. **Calibrate ZUPT thresholds (optional but recommended).** Place the phone flat on a still surface and tap **ZUPT Cal**. The app collects 200 accelerometer-magnitude samples and sets the stillness thresholds to 3× and 4× the measured mean and variance respectively. Skip this step to use the built-in defaults (mean $\leq$ 0.15 m/s^2, variance ≤ 0.006 (m/s^2)^2).
5. **Start tracking.** Hold the phone in your normal walking orientation and tap **Start**. The phone's current heading defines the world-frame x-axis; the EKF captures the initial yaw at the first rotation-vector event and uses it as ψ_0.
6. **Walk and observe.** The trajectory canvas updates in real time. The status line shows estimated (x, y), current FTM range, velocity components, heading, and bias estimates.
7. **Mark waypoints (optional, for accuracy evaluation).** At any ground-truth reference point, enter the true coordinates in the "GT:" fields and tap **Mark WP**. The waypoint log shows estimated vs. true position and Euclidean error for each marked point. All waypoints (and continuous position logs) are written to a CSV file in the app's external storage directory.
8. **Stop.** Tap **Stop** to halt ranging and flush the CSV log.
9. **Reset.** Tap **Reset** to return the EKF to its initial state without leaving the screen.

1.3 FTM Ranging Calibration

Raw FTM distances are systematically biased by hardware path delay. The companion **Calibration** activity collects range measurements at a known distance and fits a linear correction $d_{\mathrm{cal}} = \alpha\, d_{\mathrm{raw}} + \beta$. The fitted slope α and intercept β are stored in `SharedPreferences` and applied automatically in `FusionActivity` every time a ranging result arrives.

2 Overview

This document describes the Extended Kalman Filter (EKF) implemented in `FusionEstimator.kt`. The filter fuses:

- **IMU dead-reckoning** (`TYPE_LINEAR_ACCELERATION` + `TYPE_GAME_ROTATION_VECTOR`, sampled at the hardware maximum rate $\approx 200\,\text{Hz}$) in the *predict* step, and
- **Wi-Fi FTM ranging** ($\approx 20\,\text{Hz}$ after reducing the ranging interval from 100 ms to 50 ms) plus **ZUPT** (zero-velocity pseudo-measurement detected from the IMU sliding window) in the *update* step.

Motion is assumed to be two-dimensional (horizontal plane). Both body-frame acceleration axes (a_x^b and a_y^b) are used for dead-reckoning (see Section 5). The phone's initial orientation at the moment `startTracking()` is called defines the world frame.

3 State Vector

$$\mathbf{x} = \begin{bmatrix} x \\ y \\ v_x \\ v_y \\ b_x^b \\ b_y^b \end{bmatrix} \in \mathbb{R}^6$$

Symbol	Meaning
x, y	horizontal position in the world frame (metres)
v_x, v_y	horizontal velocity in the world frame (m/s)
b_x^b	accelerometer bias along the body x-axis (m/s^2)
b_y^b	accelerometer bias along the body y-axis (m/s^2)

Two bias states. Each body-frame acceleration axis carries its own independent bias. Both biases are modelled as scalar random walks; they are subtracted from the respective sensor readings *before* the body-to-world rotation so that the bias remains a body-frame constant even when the phone turns.

4 Coordinate Frames

World frame. The world frame origin is placed at the starting position. Its axes are aligned with the phone's horizontal orientation at $t = 0$: the positive x-axis points in the direction the phone's y-axis (screen top) was facing at start, the positive y-axis is 90° CCW from that.

Body frame. The sensor-native frame fixed to the phone. Both the body-x component a_x^b and body-y component a_y^b of `TYPE_LINEAR_ACCELERATION` are used in the dead-reckoning integration.

Frame rotation. `TYPE_GAME_ROTATION_VECTOR` provides a quaternion from which `SensorManager.getOrient` extracts the azimuth ψ (yaw, clockwise-positive in Android's convention). The initial azimuth ψ_0 is captured on the first rotation-vector event; thereafter the relative yaw $\phi = \psi - \psi_0$ describes how much the phone has turned since start.

Because Android's azimuth is clockwise-positive, the body-to-world rotation matrix for a 2-D projection is:

$$\mathbf{R}(\phi) = \begin{bmatrix} \cos\phi & \sin\phi \\ -\sin\phi & \cos\phi \end{bmatrix}$$

Yaw interpolation. The rotation-vector and linear-acceleration sensors run asynchronously. To evaluate ϕ at the exact accelerometer timestamp t_k, the filter stores the two most recent yaw readings $(\psi_{k-1},\, t_{k-1})$ and $(\psi_k,\, t_k)$ and linearly interpolates:

$$\phi(t) = \phi_{k-1} + \alpha\,\Delta\phi, \quad \alpha = \frac{t - t_{k-1}}{t_k - t_{k-1}} \in [0,1], \quad \Delta\phi = \text{normalizeAngle}(\psi_k - \psi_{k-1})$$

where normalizeAngle wraps the difference into $(-\pi, \pi]$ to handle the $\pm\pi$ discontinuity.

5 Body-to-World Frame Rotation

Both a_x^b and a_y^b from `TYPE_LINEAR_ACCELERATION` are transformed into the world frame using the full 2-D rotation. Both body-frame biases are subtracted *before* rotation (see Section 6 for why order matters):

$$\begin{bmatrix} a_x^w \\ a_y^w \end{bmatrix} = \begin{bmatrix} \cos\phi & \sin\phi \\ -\sin\phi & \cos\phi \end{bmatrix} \begin{bmatrix} a_x^b - b_x^b \\ a_y^b - b_y^b \end{bmatrix} = \begin{bmatrix} (a_x^b - b_x^b)\cos\phi + (a_y^b - b_y^b)\sin\phi \\ -(a_x^b - b_x^b)\sin\phi + (a_y^b - b_y^b)\cos\phi \end{bmatrix}$$

Using both axes captures centripetal acceleration during curved walking ($a_x^b = v\dot{\phi}$) and any lateral body motion, improving dead-reckoning accuracy relative to a body-y-only approximation.

6 Predict Step – Kinematic IMU Model with Body-Frame Bias

6.1 Bias Correction Order

Both biases b_x^b and b_y^b are body-frame constants. They must be subtracted from the respective sensor readings *before* the body-to-world rotation, not after. Subtracting after rotation would model the bias as a world-frame constant, which is physically incorrect: a real sensor bias rotates with the phone and therefore appears in a different world direction whenever the phone turns.

The bias-corrected body-frame accelerations are:

$$a_{x,c}^b = a_x^b - b_x^b, \qquad a_{y,c}^b = a_y^b - b_y^b$$

and the resulting world-frame accelerations are:

$$\begin{aligned} a_x^w &= a_{x,c}^b \cos\phi + a_{y,c}^b \sin\phi \\ a_y^w &= -a_{x,c}^b \sin\phi + a_{y,c}^b \cos\phi \end{aligned}$$

6.2 State Transition

$$\mathbf{x}_{k|k-1} = \mathbf{F}_k\,\mathbf{x}_{k-1|k-1} + \mathbf{B}_x\,a_{x,c}^b + \mathbf{B}_y\,a_{y,c}^b$$

$$\mathbf{B}_x = \begin{bmatrix} \frac{1}{2}\Delta t^2\cos\phi \\ -\frac{1}{2}\Delta t^2\sin\phi \\ \Delta t\cos\phi \\ -\Delta t\sin\phi \\ 0 \\ 0 \end{bmatrix}, \qquad \mathbf{B}_y = \begin{bmatrix} \frac{1}{2}\Delta t^2\sin\phi \\ \frac{1}{2}\Delta t^2\cos\phi \\ \Delta t\sin\phi \\ \Delta t\cos\phi \\ 0 \\ 0 \end{bmatrix}$$

Component-wise:

$$x \leftarrow x + v_x\Delta t + \tfrac{1}{2}\big[(a_x^b - b_x^b)\cos\phi + (a_y^b - b_y^b)\sin\phi\big]\Delta t^2$$

$$y \leftarrow y + v_y \Delta t + \tfrac{1}{2}\big[-(a_x^b - b_x^b)\sin\phi + (a_y^b - b_y^b)\cos\phi\big]\Delta t^2$$

$$v_x \leftarrow v_x + \big[(a_x^b - b_x^b)\cos\phi + (a_y^b - b_y^b)\sin\phi\big]\Delta t$$

$$v_y \leftarrow v_y + \big[-(a_x^b - b_x^b)\sin\phi + (a_y^b - b_y^b)\cos\phi\big]\Delta t$$

$$b_x^b \leftarrow b_x^b, \qquad b_y^b \leftarrow b_y^b \quad \text{(random walks — driven by process noise only)}$$

6.3 State Transition Jacobian

$\mathbf{F}_k$ is the 6×6 Jacobian of the state update. The two bias columns are obtained by differentiating with respect to b_x^b (column 4) and b_y^b (column 5):

$$\frac{\partial a_x^w}{\partial b_x^b} = -\cos\phi, \quad \frac{\partial a_y^w}{\partial b_x^b} = +\sin\phi; \qquad \frac{\partial a_x^w}{\partial b_y^b} = -\sin\phi, \quad \frac{\partial a_y^w}{\partial b_y^b} = -\cos\phi$$

Therefore:

$$\frac{\partial x'}{\partial b_x^b} = -\tfrac{\Delta t^2}{2}\cos\phi, \quad \frac{\partial y'}{\partial b_x^b} = +\tfrac{\Delta t^2}{2}\sin\phi, \quad \frac{\partial v_x'}{\partial b_x^b} = -\Delta t\cos\phi, \quad \frac{\partial v_y'}{\partial b_x^b} = +\Delta t\sin\phi$$

$$\frac{\partial x'}{\partial b_y^b} = -\tfrac{\Delta t^2}{2}\sin\phi, \quad \frac{\partial y'}{\partial b_y^b} = -\tfrac{\Delta t^2}{2}\cos\phi, \quad \frac{\partial v_x'}{\partial b_y^b} = -\Delta t\sin\phi, \quad \frac{\partial v_y'}{\partial b_y^b} = -\Delta t\cos\phi$$

The full 6×6 Jacobian is therefore **time-varying** (it depends on the current yaw ϕ):

$$\mathbf{F}_k = \begin{bmatrix} 1 & 0 & \Delta t & 0 & -\frac{\Delta t^2}{2}\cos\phi_k & -\frac{\Delta t^2}{2}\sin\phi_k \\ 0 & 1 & 0 & \Delta t & +\frac{\Delta t^2}{2}\sin\phi_k & -\frac{\Delta t^2}{2}\cos\phi_k \\ 0 & 0 & 1 & 0 & -\Delta t\cos\phi_k & -\Delta t\sin\phi_k \\ 0 & 0 & 0 & 1 & +\Delta t\sin\phi_k & -\Delta t\cos\phi_k \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

The values $\sin\phi_k$ and $\cos\phi_k$ are computed once per IMU event (from the interpolated yaw) and forwarded to `predictKinematic` so they are not recomputed during covariance propagation.

6.4 Process Noise Covariance

$\mathbf{Q}$ has two independent components:

Kinematic block. Continuous-time white acceleration noise with standard deviation σ_a, discretised over Δt:

$$\mathbf{Q}_{\text{kin}} = \sigma_a^2 \begin{bmatrix} \frac{\Delta t^4}{4} & 0 & \frac{\Delta t^3}{2} & 0 & 0 & 0 \\ 0 & \frac{\Delta t^4}{4} & 0 & \frac{\Delta t^3}{2} & 0 & 0 \\ \frac{\Delta t^3}{2} & 0 & \Delta t^2 & 0 & 0 & 0 \\ 0 & \frac{\Delta t^3}{2} & 0 & \Delta t^2 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

Bias random-walks. Each bias drifts independently as a scalar random walk:

$$\mathbf{Q}_{\text{bias}} = \operatorname{diag}\big(0,\, 0,\, 0,\, 0,\, \sigma_{b_x}^2\Delta t,\, \sigma_{b_y}^2\Delta t\big)$$

The full process noise is $\mathbf{Q} = \mathbf{Q}_{\text{kin}} + \mathbf{Q}_{\text{bias}}$.

ZUPT suppression. When the device is confirmed stationary (see Section 8), the entire process noise $\mathbf{Q} = \mathbf{Q}_{\text{kin}} + \mathbf{Q}_{\text{bias}}$ is suppressed (set to zero). Suppressing only the kinematic block while leaving the bias random walk active would continuously inflate $P_{b_x b_x}$ and $P_{b_y b_y}$; the time-varying $\mathbf{F}_k$ then propagates that uncertainty into the position rows via the $\partial x'/\partial b_x^b$ coupling terms, raising P_{xx} and P_{yy} even during standstill and producing large Kalman corrections on the next FTM measurement — visible as a position jump. Suppressing all of $\mathbf{Q}$ prevents $\mathbf{P}$ from growing while stationary; the ZUPT velocity update simultaneously drives velocity uncertainty down. The time-varying $\mathbf{F}_k$ is still evaluated with the current $\sin\phi_k$, $\cos\phi_k$ so that the FPF^T propagation of existing off-diagonal terms remains correct.

6.5 Covariance Propagation

$$\mathbf{P}_{k|k-1} = \mathbf{F}_k \, \mathbf{P}_{k-1|k-1} \, \mathbf{F}_k^\top + \mathbf{Q}$$

$\mathbf{F}_k$ has at most 4 non-zeros per row; the product $\mathbf{F}_k \mathbf{P} \mathbf{F}_k^\top$ is computed in two sparse passes without forming the full 6×6 matrix product. Pre-allocated scratch buffers avoid heap allocation on the 200 Hz hot path. Diagonal entries are capped at $P_{\max} = 100\,\text{m}^2$.

Parameter	Value
σ_a	$0.3\ \text{m/s}^2$
σ_{b_x}	$0.01\ \text{m/s}^2/\sqrt{\text{s}}$
σ_{b_y}	$0.01\ \text{m/s}^2/\sqrt{\text{s}}$
$P_{\max}$	$100\ \text{m}^2$
$\Delta t_{\max}$	0.5 s (samples with larger gap discarded)

7 Update Step 1 – Wi-Fi FTM Range

7.1 Measurement Model

The FTM chip measures the *3-D slant range* between the phone and the access point. The vertical offset is

$$\Delta z = z_{\text{phone}} - z_{\text{AP}}$$

(default $\Delta z = 1.2\,\text{m}$; set via the "Δz" field in the UI).

The nonlinear measurement function is

$$h(\mathbf{x}) = \sqrt{(x - x_{\text{AP}})^2 + (y - y_{\text{AP}})^2 + \Delta z^2}$$

7.2 Jacobian

$$\mathbf{H}_{\text{ftm}} = \frac{\partial h}{\partial \mathbf{x}} = \left[\frac{x - x_{\text{AP}}}{r_{3D}},\ \frac{y - y_{\text{AP}}}{r_{3D}},\ 0,\ 0,\ 0,\ 0 \right] \in \mathbb{R}^{1\times 6}$$

where $r_{3D} = h(\mathbf{x})$. Velocity and bias components are zero because range is independent of both. Both biases are nonetheless updated indirectly: $\mathbf{P}$ develops non-zero off-diagonal terms between b_x^b, b_y^b and the position rows through the time-varying $\mathbf{F}_k$ coupling, so the Kalman gains $K_{b_x^b}$ and $K_{b_y^b}$ are generally non-zero and FTM corrections propagate into both bias estimates.

7.3 Kalman Update

$$S = \mathbf{H}_{\text{ftm}}\,\mathbf{P}\,\mathbf{H}_{\text{ftm}}^{\top} + \sigma_{\text{ftm}}^2 \qquad \in \mathbb{R}$$

$$\mathbf{K} = \frac{\mathbf{P}\,\mathbf{H}_{\text{ftm}}^{\top}}{S} \qquad \in \mathbb{R}^{6\times 1}$$

$$\nu = d_{\text{meas}} - r_{3D}, \qquad \mathbf{x} \leftarrow \mathbf{x} + \mathbf{K}\,\nu, \qquad \mathbf{P} \leftarrow (\mathbf{I} - \mathbf{K}\,\mathbf{H}_{\text{ftm}})\,\mathbf{P}$$

The $\mathbf{P}$ update is computed over the upper triangle only and mirrored; rows 0 and 1 are saved before the in-place loop to prevent read-after-write corruption.

7.4 FTM Pre-processing

Raw FTM distances are smoothed with a 5-sample causal moving average before being passed to the EKF. At the 50 ms ranging interval (20 Hz) this corresponds to a 250 ms smoothing window.

Parameter	Value
σ_{ftm}	0.25 m
Outlier gate $\Delta d_{\max}$	3.0 m jump between consecutive readings
Ranging interval	50 ms (20 Hz)
Moving-average window	5 samples (250 ms)

8 Update Step 2 – ZUPT (Zero-Velocity Pseudo-Measurement)

8.1 Threshold Calibration

Before tracking begins, the user may tap the **ZUPT Cal** button while holding the phone stationary. The activity collects $N = 200$ raw 3-D acceleration-magnitude samples $\|[a_x^b,\, a_y^b,\, a_z^b]\|$ and passes them to `calibrateZupt()`, which sets:

$$\bar{m}_{\text{thresh}} = \max\!\big(0.05,\; 3\,\bar{m}_{\text{cal}}\big), \qquad \sigma_{m,\text{thresh}}^2 = \max\!\big(0.001,\; 4\,\hat{\sigma}_{m,\text{cal}}^2\big)$$

where $\bar{m}_{\text{cal}}$ and $\hat{\sigma}_{m,\text{cal}}^2$ are the sample mean and variance of the collected magnitudes. The $3\times$ and $4\times$ factors provide headroom between the noise floor and the detection thresholds. If calibration is skipped the default values listed below are used.

8.2 Detection

The ZUPT detector uses a sliding window of the last 25 bias-corrected world-frame acceleration magnitudes $\|[a_x^w,\, a_y^w]\|$. The device is declared *still* when:

1. the window mean $\bar{m} < \bar{m}_{\text{thresh}}$, *and*
2. the window variance $\sigma_m^2 < \sigma_{m,\text{thresh}}^2$.

Stillness is confirmed after 10 consecutive frames satisfying these criteria.

8.3 Measurement Model

When confirmed still, we assert that the true velocity is zero:

$$\mathbf{H}_{\text{zupt}} = \begin{bmatrix} 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \end{bmatrix} \in \mathbb{R}^{2\times 6}, \qquad \mathbf{R}_{\text{zupt}} = \sigma_{\text{zupt}}^2\,\mathbf{I}_2$$

8.4 Kalman Update

$$\mathbf{S}_{\text{zupt}} = \begin{bmatrix} P_{v_x v_x} + \sigma_{\text{zupt}}^2 & P_{v_x v_y} \\ P_{v_x v_y} & P_{v_y v_y} + \sigma_{\text{zupt}}^2 \end{bmatrix} \in \mathbb{R}^{2\times 2}$$

The 2×2 innovation covariance is inverted analytically.

$$\mathbf{K}_{\text{zupt}} = \mathbf{P}\,\mathbf{H}_{\text{zupt}}^{\top}\,\mathbf{S}^{-1} \in \mathbb{R}^{6\times 2}$$

$\mathbf{P}\,\mathbf{H}_{\text{zupt}}^{\top}$ is columns 2 and 3 of $\mathbf{P}$, so no general matrix product is needed.

$$\boldsymbol{\nu}_{\text{zupt}} = -\begin{bmatrix} v_x \\ v_y \end{bmatrix}, \qquad \mathbf{x} \leftarrow \mathbf{x} + \mathbf{K}_{\text{zupt}}\,\boldsymbol{\nu}_{\text{zupt}}, \qquad \mathbf{P} \leftarrow (\mathbf{I} - \mathbf{K}_{\text{zupt}}\,\mathbf{H}_{\text{zupt}})\,\mathbf{P}$$

Rows 2 and 3 of $\mathbf{P}$ are saved before the in-place loop. During ZUPT the predict step is called with $a_{x,c}^b = a_{y,c}^b = 0$ (zero world-frame acceleration) and $\mathbf{Q} = \mathbf{0}$ (both kinematic and bias noise suppressed).

Parameter	Value
σ_{zupt}	0.05 m/s
Window size	25 samples
Confirmation frames	10
Mean threshold (default)	0.15 m/s^2 (override via ZUPT Cal)
Variance threshold (default)	0.006 (m/s^2)^2 (override via ZUPT Cal)

9 Initialisation

1. **With a known start position** (x_0, y_0)**:** state is set to $(x_0, y_0, 0, 0, 0, 0)$,

   $$\mathbf{P}_0 = \text{diag}(0.25,\ 0.25,\ 0.01,\ 0.01,\ 0.01,\ 0.01)$$

   (units: m^2, m^2, (m/s)^2, (m/s)^2, (m/s^2)^2, (m/s^2)^2). On the first FTM reading the position is snapped onto the correct 2-D circle of radius $\sqrt{d^2 - \Delta z^2}$ around the AP.

2. **Without a start position:** state $\mathbf{x}$ is zeroed, $\mathbf{P}_0 = \text{diag}(9,\ 9,\ 0.01,\ 0.01,\ 0.01,\ 0.01)$. Only $r_{3\text{D}}$ is reported until a start position is provided.

10 Sequence of Operations (per IMU event)

1. Receive `TYPE_LINEAR_ACCELERATION` (a_x^b, a_y^b) at time t_k.
2. Interpolate yaw ϕ_k; compute $\sin\phi_k$, $\cos\phi_k$.
3. Subtract body-frame biases: $a_{x,c}^b = a_x^b - b_x^b$, $a_{y,c}^b = a_y^b - b_y^b$.
4. Rotate both bias-corrected axes to world frame:

   $$a_x^w = a_{x,c}^b \cos\phi_k + a_{y,c}^b \sin\phi_k, \quad a_y^w = -a_{x,c}^b \sin\phi_k + a_{y,c}^b \cos\phi_k$$

5. Update ZUPT sliding window with $\|[a_x^w, a_y^w]\|$; check stillness criteria.
6. If confirmed still: call `predictKinematic`(Δt, 0, 0, $\sin\phi_k$, $\cos\phi_k$, `suppressQ=true`), then call `updateZupt()`.
7. Else: call `predictKinematic`(Δt, a_x^w, a_y^w, $\sin\phi_k$, $\cos\phi_k$).

Per FTM event (asynchronous, $\approx 20\,\mathrm{Hz}$):

1. Apply calibration: $d_{\mathrm{cal}} = \alpha\, d_{\mathrm{raw}} + \beta$.
2. Apply 5-sample causal moving average.
3. Outlier gate: discard if $|d - d_{\mathrm{prev}}| > 3\,\mathrm{m}$.
4. Call `updateFtm`(d_{cal}).